SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Programme: **B.E / B.Tech**

Course Code: **CSA17**

Course Name: **ARTIFICIAL INTELLIGENCE**

Submission Date: 04-11**-2026**

Date of Issue :28**-10-2026** Max Marks: **50 Marks**

ASSIGNMENT – COURSE OUTCOME 1

Course Outcomes

On successful completion of the course, students will be able to:

CO	Course Outcome Statement
CO1	Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.

Evidence Rule Common to All Assignments

Every implemented task must be supported by: **Source Code → Test Input → Expected Behaviour → Actual Output → Performance Metrics (table/graph over at least three input sizes or seeds) → Interpretation.** A screenshot showing only a final answer is not sufficient. Operation counts and task metrics are primary evidence; execution time is secondary. Wherever a task says *Predict*, the prediction must be written before the experiment is run. Programming language: Python 3 (Jupyter / Colab).

CO1: Foundation and history of AI, intelligent agents (agents and environments, rationality, nature of environments, structure of agents), problem-solving agents, problem formulation, searching for solutions, uninformed search strategies and search with partial information.

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Warehouse Picking Robot **(Total Marks – 50)**

Scenario — Warehouse Picking Robot

A large e-commerce fulfilment centre uses autonomous mobile robots to bring shelf racks from storage bays to packing stations while thousands of orders arrive every hour during a festival sale. The warehouse floor is modelled as a graph of locations (aisle junctions, cross-aisles, lift points, packing-station gates). The robot may move between connected locations and every move has a travel-time cost; aisles may be temporarily occupied by other robots or by staff restocking shelves.

As new storage halls and mezzanine floors are added, the state space of the robot may grow:
1,000 → 10,000 → 100,000 → 1,000,000+ states

You are required to investigate, design and compare uninformed search strategies by which the robot finds a route from its charging dock to a requested storage bay as the warehouse grows.

Your solution must remain practical for both small and very large state spaces. Any preprocessing (for example, building adjacency lists or a duplicate-detection structure) must be included in your efficiency analysis.

Design Challenge (Day 4): The robot's floor-marker (QR-code) reader now fails intermittently, so the robot no longer knows exactly where it is. Re-evaluate your original design and decide whether it should be retained, modified or redesigned.

TASK 1 – Design the Agent and the Search Algorithms (5 Marks)

Before designing algorithms, define the agent and the problem:

- Specify the PEAS description of the picking robot and classify its environment (observable, deterministic, episodic/sequential, static/dynamic, discrete/continuous, single/multi-agent), justifying each choice
- State which view of AI (acting rationally, thinking rationally, acting humanly, thinking humanly) the robot follows and why it is a rational problem-solving agent. Relate the problem type briefly to the early history of AI problem solving
- Formulate the problem: states, initial state, actions, transition model, goal test, path cost

Then design and propose at least **three meaningfully different** uninformed search strategies for finding a route. Iterative and recursive implementations of the same underlying strategy will not be considered two different approaches.

Your proposed approaches must include:

- At least one iterative approach (queue- or stack-based)
- At least one recursive approach
- A clear explanation of the basic idea behind each approach
- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required, such as adjacency-list construction or an explored set

You are expected to select the approaches independently. The appropriateness and diversity of the selected strategies are part of the assessment.

Evidence Expected: PEAS and environment classification, complete problem formulation, three distinct strategies with pseudocode/flowcharts, preprocessing requirements and justification for each.

TASK 2 – Identify the Basic Operation (2 Marks)

For each proposed algorithm, identify the basic operation that predominantly determines its execution time.

Discuss:

- What the basic operation is (for example, node expansion, successor generation, goal test or frontier insertion)
- Where it occurs in the algorithm
- Why it is appropriate for efficiency analysis
- How the number of times it executes may change as the state-space size increases

Do not simply name an operation. Relate it to the behaviour of the algorithm.

Evidence Expected: Correct identification and justification of the basic operation for each algorithm.

TASK 3 – Derive the Time- and Space-Complexity Functions Mathematically (6 Marks)

Derive the time and space complexity for each proposed approach using mathematical analysis.

Discuss:

- The input size (branching factor b, shallowest-goal depth d, maximum depth m, optimal cost C^*, minimum step cost ε)
- The number of times the basic operation executes, derived by summing the nodes at each level
- The mathematical expression for time and for memory (frontier size)
- Any preprocessing cost involved
- The overall cost when preprocessing and searching are considered together

You must show the mathematical reasoning. Merely stating that an algorithm is O(b^d) will not receive full credit.
Evidence Expected: A clear derivation of time and space functions for each approach, including operation counts and preprocessing costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour (4 Marks)

Analyse each algorithm under different input conditions.
Discuss, wherever applicable:

- What input condition produces the best case (for example, the goal lies on the first branch explored)
- What condition produces the average case
- What condition produces the worst case (for example, goal at maximum depth, goal unreachable, cheapedge detours)
- How the number of nodes generated changes in each case, and why

Do not simply provide a table of complexity values. Explain the conditions that lead to each case.
Evidence Expected: A justified best-, average- and worst-case analysis supported by mathematical reasoning.

TASK 5 – Perform Asymptotic Analysis (3 Marks)

Express and compare the growth behaviour of the algorithms using O, Ω and/or Θ.
Discuss:

- How each algorithm behaves as the state space becomes very large
- How the growth rates of the approaches differ in time and in memory
- What the growth means when the warehouse grows from 1,000 to 1,000,000+ states, for example, whether memory or time becomes the limiting factor first

Interpret the behaviour in the context of the warehouse picking robot rather than merely stating the notation.
Evidence Expected: Correct asymptotic representation and a meaningful comparison of growth behaviour.

TASK 6 – Formulate the Recurrence Relation (3 Marks)

For the recursive approach, formulate a recurrence relation directly from the behaviour of your algorithm.
Identify clearly:

- The size of the original problem (depth limit or remaining depth)
- The size of the recursive subproblem
- The number of recursive calls (related to the branching factor)
- The work performed outside the recursive calls
- The base case
- How these components lead to the recurrence

The recurrence must be derived from your algorithm, not copied from a textbook.
Evidence Expected: A correct recurrence with clear identification of its components and base case.

TASK 7 – Solve the Recurrence Relation (4 Marks)

Solve the recurrence from Task 6 using an appropriate analytical method (substitution, iteration, recursion tree or another method).
Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the recursive algorithm as depth or branching factor increases
- How the result extends to iterative deepening, where the recursive search is repeated for increasing limits

Show the analytical process, not only the final result.

Evidence Expected: A complete solution with justification of the method and interpretation of the result.

TASK 8 – Implement and Test the Algorithms (4 Marks)

Implement all proposed algorithms in Python and test them experimentally.
Use generated warehouse graphs of approximately:
1,000 → 10,000 → 100,000 → 1,000,000 states
Ensure that:

- All algorithms return correct routes
- The same or equivalent graphs and start/goal pairs are used for comparison
- Testing conditions are consistent and the random seeds are reported
- Test cases include a reachable goal, an unreachable goal and a start equal to the goal
- The implementation corresponds to the algorithms analysed in Tasks 1–7

The purpose of implementation is to generate evidence for validating your theoretical analysis.
Evidence Expected: Working code, test inputs, expected behaviour and actual outputs, with a reproducible testing procedure.

TASK 9 – Record Experimental Performance (3 Marks)

Collect and present performance data for each algorithm across the different state-space sizes.
Record, with code and outputs as proof:

- Nodes generated and nodes expanded
- Maximum frontier size (memory proxy)
- Solution depth and path cost, and whether the path is optimal
- Execution time (averaged over at least 5 runs)

Present the results using tables and graphs (use a log scale where appropriate) so that approaches can be compared easily.
Evidence Expected: Systematically collected experimental data with clear tables and graphs.

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

Compare your theoretical analysis with the experimental results.
Discuss:

- Whether the observed growth of nodes generated and of frontier size agrees with the theoretical complexity
- Which algorithm scales better as the state space increases
- Any differences between theory and observation, and the possible reasons (graph structure, duplicates, tie-breaking, language overhead)

Interpret the results rather than simply reproducing tables or graphs.
Evidence Expected: A reasoned comparison between theoretical and observed performance.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

Based on your mathematical analysis and experimental findings, recommend the most appropriate search strategy for the warehouse picking robot.
Discuss:

- Which approach you recommend and why it is more appropriate than the alternatives
- Its theoretical efficiency and experimentally observed performance
- Any preprocessing cost, trade-offs or limitations
- Whether it remains practical as the warehouse grows

Your recommendation must be based on your own evidence. Simply choosing the algorithm with the lowest BigO complexity is not sufficient.
Evidence Expected: A technically justified recommendation supported by analysis and experiment.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

After a power restart or a floor-marker reader fault, the robot no longer knows exactly where it is. It knows only that it is at one of *k* possible aisle junctions. It must still reach its charging dock, whichever junction it is actually at.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

- What has changed from the original problem and which original assumptions (fully observable, known start state) are affected?
- Why does the robot's knowledge now form part of the state?

B. Evaluation of the Original Design – 2 Marks

- Can your original algorithm still be applied? What does a "state" become?
- How large can the new search space be for *k* possible locations, and is the original design acceptable?

C. Adaptation or Redesign – 2 Marks

- Describe the revised strategy (for example, search over belief states, or search with local sensing and a conditional plan), what has changed and the effect on complexity
- If you retain the original solution, give sufficient technical justification

D. Evidence-Based Defence – 2 Marks

- Defend your decision using mathematical analysis and a small experiment (belief states generated and plan length for *k* = 2, 4, 8, with and without a local sensor), with code and output as proof

RUBRICS FOR EVALUATION

Task / Criteria	Marks	Level 4 – Exemplary (90–100%)	Level 3 – Proficient (70–89%)	Level 2 – Developing (40–69%)	Level 1 – Beginning (0–39%)
Task 1 – Agent design and search strategies	5	Scenario-specific PEAS; all six properties justified; AI view and history linked; complete formulation; three distinct strategies (iterative and recursive) with pseudocode, preprocessing and justification.	PEAS and formulation complete with minor gaps; three strategies with pseudocode, justification brief.	Generic PEAS; formulation incomplete; strategies not distinct or pseudocode missing.	Agent or formulation missing; fewer than two strategies.
Task 2 – Basic operation	2	Correct basic operation for every algorithm, located in the code and related to algorithm behaviour and growth.	Correct operations with brief justification.	Operations named without justification.	Incorrect or missing.
Task 3 – Complexity derivation	6	Level-by-level summation for time and frontier memory of each approach; preprocessing cost included; correct combined cost.	Derivations mostly complete; minor slips or preprocessing omitted.	Big-O stated with little derivation.	No derivation.

Task 4 – Best, average and worst case	4	Clear input conditions for each case and algorithm, with nodecount reasoning.	Cases correct; reasoning partly given.	Table of values without conditions.	Missing or incorrect.
Task 5 – Asymptotic analysis	3	Correct O/Ω/Θ for time and memory; growth from 1,000 to 1,000,000+ states interpreted for the robot.	Correct notation; limited interpretation.	Notation partly wrong; no interpretation.	Missing.
Task 6 – Recurrence formulation	3	Recurrence derived from own recursive algorithm with problem size, subproblems, calls, outside work and base case identified.	Correct recurrence; one component unclear.	Recurrence copied or components missing.	Missing or wrong.
Task 7 – Recurrence solution	4	Method justified; all steps shown; correct result interpreted and extended to iterative deepening.	Correct solution with minor gaps in steps or interpretation.	Final answer only, or method misapplied.	Missing or wrong.
Task 8 – Implementation and testing	4	All algorithms working on all four graph sizes; seeds reported; reachable, unreachable and start = goal cases tested; code matches analysis.	Working code; one test case type or size missing.	Partial implementation or untested.	No working code.
Task 9 – Performance recording	3	Nodes generated/expanded, max frontier, depth, cost, optimality and averaged time recorded for all sizes, with clear tables and log-scale graphs.	Most metrics recorded; graphs basic.	Few metrics; no graphs.	No data.
Task 10 – Theory vs experiment	4	Observed growth compared with theory for every algorithm; differences explained with sound reasons.	Comparison made; reasons partly explained.	Tables repeated with little interpretation.	Missing.
Task 11 – Recommendation	4	Recommendation backed by own analysis and evidence; tradeoffs, preprocessing and scalability discussed.	Reasonable recommendation; evidence partly used.	Recommendation based only on Big-O.	Missing or unjustified.
Task 12 – Day-4 re-evaluation	8	Impact analysis, evaluation, adaptation (belief states / conditional plan) and defence all complete, with experiment for k = 2, 4, 8 with and without a local sensor.	Three of four parts strong; experiment partial.	Generic discussion; no experiment.	Missing.

Total: 50 marks. Each task is marked out of the marks shown by choosing the level that best matches the evidence submitted and awarding the matching percentage of that task's marks (round to the nearest 0.5). A task without source code, test input and actual output as proof (Evidence Rule) cannot score above Level 2.

CODE OF CONDUCT

I A.Khaja Hussain (192572118) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assignment. I understand that any violation of academic integrity rules will result in disciplinary action

Signature of the student: A.khaja Hussain

1. PEAS Description of Warehouse Picking Robot

PEAS stands for Performance Measure, Environment, Actuators and Sensors.

PEAS	Description
Performance Measure	Reach the required storage location safely, reduce travel time and avoid collisions.
Environment	Warehouse floor, storage racks, aisles, charging dock and packing stations.
Actuators	Wheels, motors, steering system and robotic arms.
Sensors	QR-code reader, cameras, distance sensors and obstacle detectors.

2. Classification of the Environment

1. Partially Observable: The robot may not always know the complete warehouse condition because other robots or workers can block its sensors.
2. Stochastic: The environment may change unexpectedly due to moving obstacles and temporary aisle blockages.
3. Sequential: Every movement affects the robot's next movement and final destination.
4. Dynamic: The warehouse changes continuously because robots and workers move around.
5. Discrete: Warehouse locations and movements are represented as separate nodes and connections in a graph.
6. Multi-Agent: Multiple robots and human workers operate in the same warehouse environment.

3. View of Artificial Intelligence

The robot follows the Acting Rationally approach.

It selects the best possible action based on available information to reach the destination safely and efficiently. It is called a rational problem-solving agent because it chooses actions that minimize travel cost and achieve its goal.

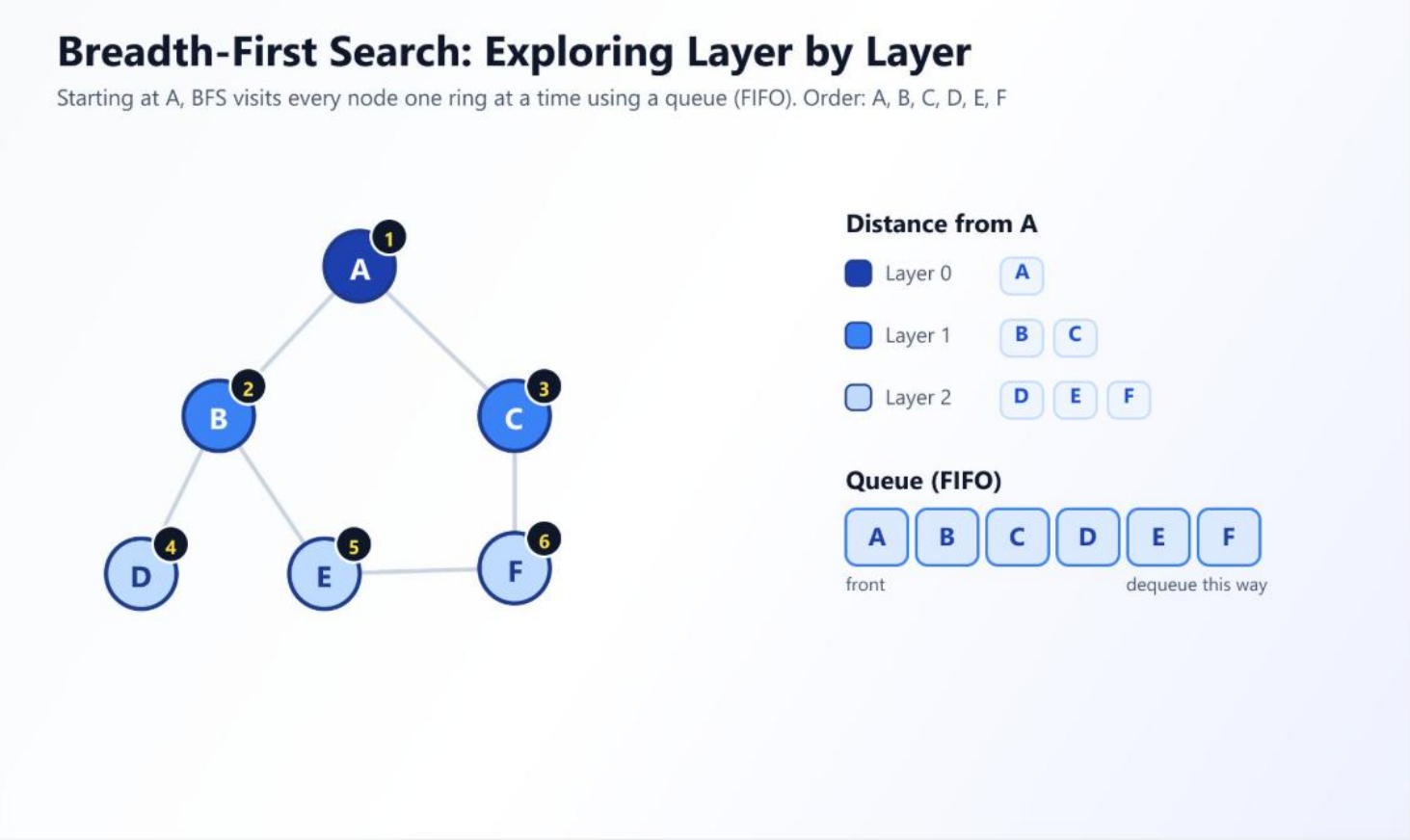
Early AI research focused on problem-solving techniques, logical reasoning and search algorithms. These techniques are used in the warehouse robot to find a suitable route from one location to another.

4. Problem Formulation

- Initial State: Robot starts at its charging dock.
- States: Different warehouse locations such as aisles, junctions and storage bays.
- Actions: Move up, down, left, right or to another connected location.
- Transition Model: Describes the next location reached after performing an action.
- Goal Test: Checks whether the robot has reached the required storage bay.
- Path Cost: Total travel time required to reach the destination.

5. Three Uninformed Search Strategies

A. Breadth-First Search (BFS)



Breadth-First Search explores the nodes level by level. It first visits all nearby locations and then moves to the next level. It uses a FIFO queue to store the nodes that need to be explored.

Advantages:

- Finds the shortest path when all movements have equal costs.
- Complete when the branching factor is finite.
- Easy to implement and understand.

Disadvantages:

- Requires more memory.
- Becomes expensive for large warehouse environments.
- Does not guarantee the lowest travel time when movement costs differ.

Pseudocode:

```
BFS(start, goal):
    Create an empty queue
    Insert start into queue
    Mark start as visited

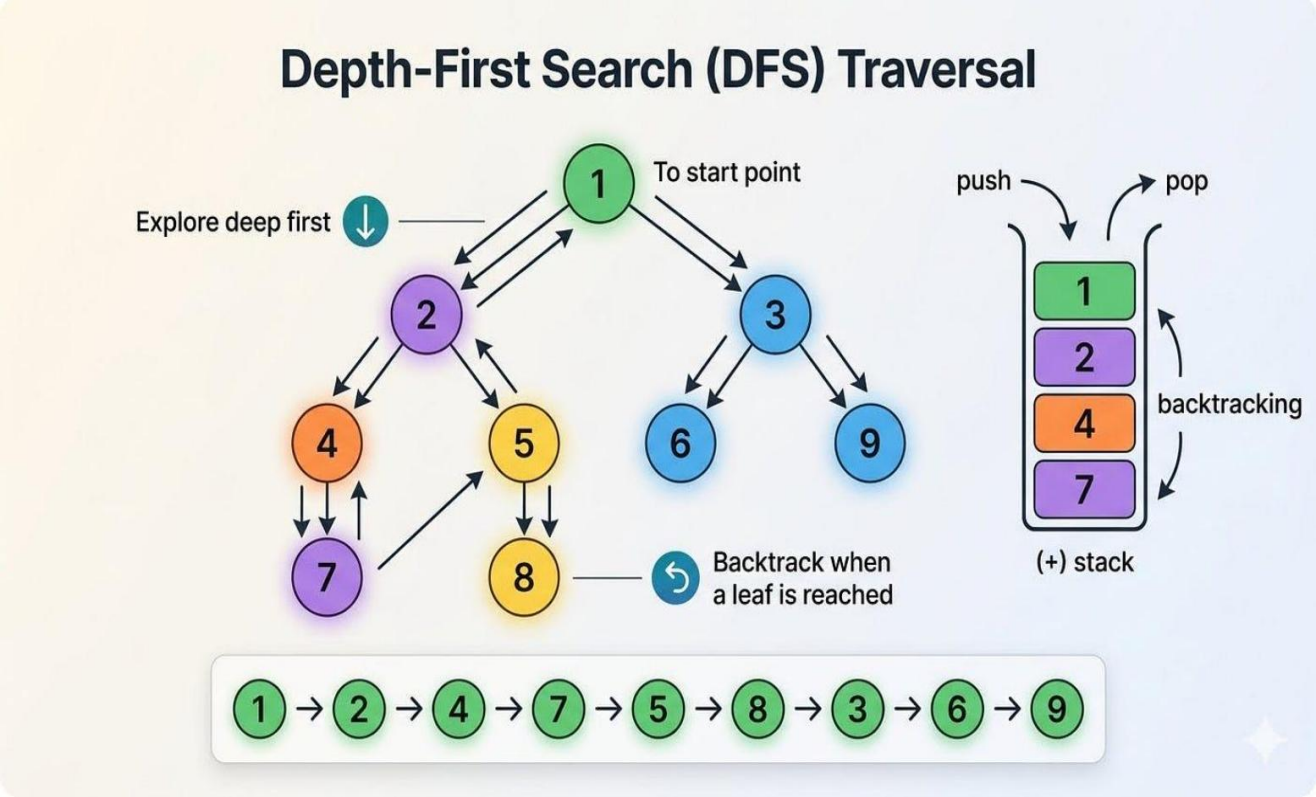
    While queue is not empty:
        Remove the first node

        If node is goal:
```

Return path

For each neighbour:
If neighbour is not visited:
Mark neighbour as visited
Insert neighbour into queue

Return failure



Depth-First Search explores one path as deeply as possible before returning to explore another path. It uses recursion or a stack to explore the warehouse locations.
Advantages:

- Requires less memory than BFS in many cases.
- Simple recursive implementation.
- Can find a solution without exploring every level.

Disadvantages:

- May explore a very deep path before finding the goal.
- Does not guarantee the shortest path.
- Can get stuck without cycle detection or a depth limit.

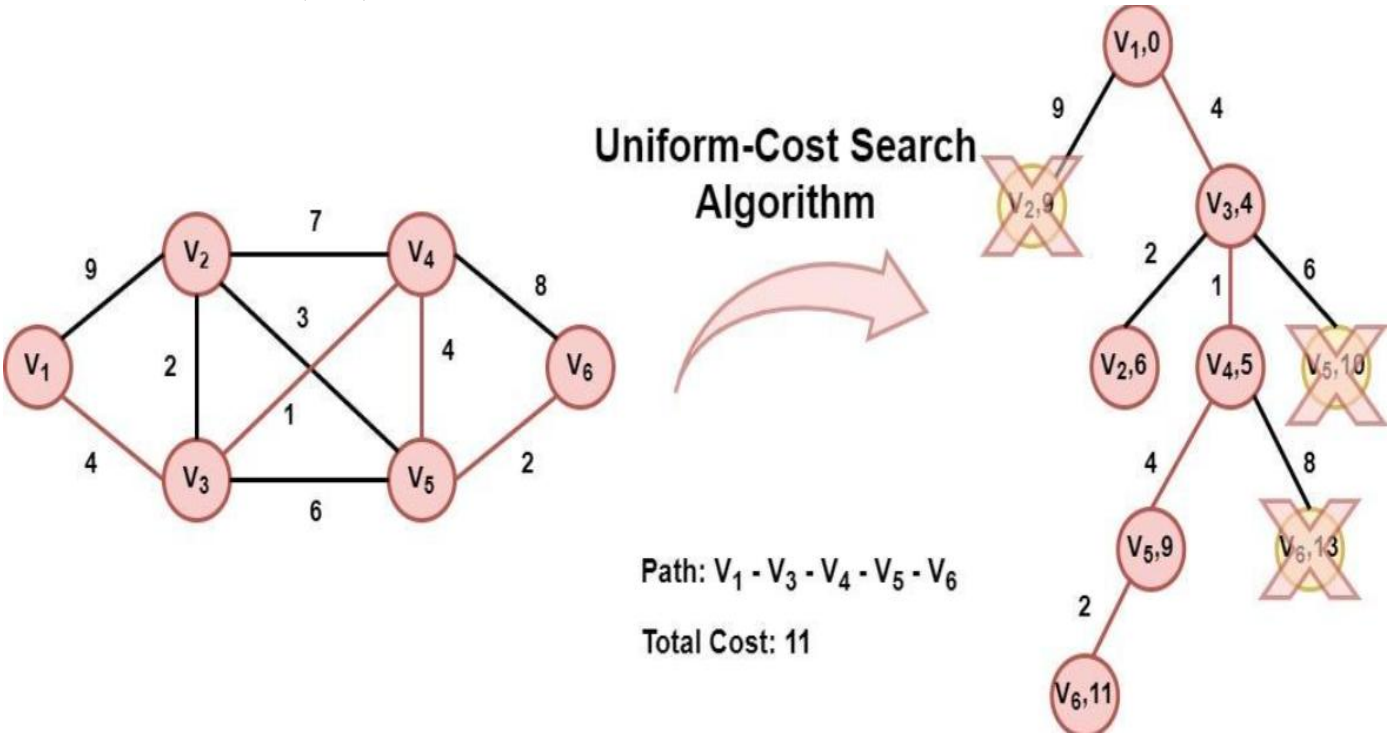
Pseudocode:
DFS(node, goal):
If node is goal:
Return success

Mark node as visited

For each neighbour of node:
If neighbour is not visited:
If DFS(neighbour, goal) is success:
Return success

Return failure

C. Uniform-Cost Search (UCS)



Uniform-Cost Search expands the node with the lowest total path cost. It uses a priority queue to select the next location based on travel time.

Advantages:

- Finds the minimum-cost path.
- Considers different travel times between locations.
- Suitable for weighted warehouse graphs when step costs are positive.

Disadvantages:

- Requires more memory for the priority queue.
- Can expand many nodes before finding the destination.
- Takes more processing time than simpler searches in some situations.

Pseudocode:

UCS(start, goal):
Create a priority queue
Insert start with cost 0

While queue is not empty:
Remove node with lowest cost

```
    If node is goal:
        Return path

    For each neighbour:
        Calculate new path cost

        If new cost is lower:
            Update cost
            Insert neighbour into queue

Return failure
```

6. Preprocessing Required

Preprocessing prepares the warehouse graph before searching.

1. Convert warehouse locations into graph nodes.
2. Store connected locations using adjacency lists.
3. Assign travel-time costs to each connection.
4. Create a visited set to avoid repeated exploration.
5. Maintain parent information to reconstruct the final route.

Justification: BFS is selected for level-wise exploration, recursive DFS for depth-based exploration and UCS for finding minimum travel-time routes. These approaches provide different search behaviours for comparison.

TASK 2 – Identify the Basic Operation

The basic operation is the main operation that determines the execution time of an algorithm.

Algorithm	Basic Operation	Explanation
BFS	Node expansion	Removes a node from the queue and checks its neighbours.
DFS	Recursive node expansion	Visits a node and recursively explores its unvisited neighbours.
UCS	Priority queue removal and relaxation	Removes the lowest-cost node and checks whether a cheaper route to its neighbours exists.

Explanation:

1. In BFS, node expansion happens whenever a node is removed from the queue. As the warehouse grows, more nodes may need to be expanded.

2. In DFS, recursive calls explore connected locations. The number of calls increases when the search goes deeper.
3. In UCS, priority queue operations and edge relaxation determine the execution time. More connections and different path costs increase the number of operations.

   These operations are suitable for analysing efficiency because they directly measure the amount of work performed by each algorithm.

TASK 3 – Derive the Time and Space Complexity Mathematically

1. Input Parameters

- b = Branching factor (average number of successors).
- d = Depth of the shallowest goal.
- m = Maximum depth of the search tree.
- C^* = Optimal solution cost.
- ϵ = Minimum positive step cost.

2. Breadth-First Search

BFS explores nodes level by level.

Number of nodes at each level:

- Level 0 = 1
- Level 1 = b
- Level 2 = b^2
- Level 3 = b^3

Total nodes explored up to depth d:

$$ N=1+b+b^2+\cdots+b^d $$

Using the geometric series:

$$ N=\frac{b^{d+1}-1}{b-1} $$

Therefore:

Time Complexity:

$$ T(b,d)=O(b^d) $$

Space Complexity:

$$ S(b,d)=O(b^d) $$

BFS requires more memory because it stores many unexplored nodes in its queue.

3. Depth-First Search

DFS explores one branch deeply before backtracking.

Total nodes in the worst-case search tree:

$$ N=1+b+b^2+\cdots+b^m $$

$$ N=\frac{b^{m+1}-1}{b-1} $$

Therefore:
Time Complexity:
$$T(b,m)=O(b^m)$$
Space Complexity:
$$S(b,m)=O(bm)$$
DFS uses less memory because it stores the current path and a limited number of unexplored alternatives.
4. Uniform-Cost Search
UCS expands nodes according to their accumulated path costs.
The maximum depth of a solution within optimal cost C^* is bounded by:
$$d_{\max}=\left\lfloor\frac{C^*}{\epsilon}\right\rfloor$$
The number of generated nodes can be bounded using:
$$N=1+b+b^2+\cdots+b^{\lfloor C^*/\epsilon\rfloor}$$
Therefore:
Time Complexity:
$$T=O\left(b^{1+\lfloor C^*/\epsilon\rfloor}\right)$$
Space Complexity:
$$S=O\left(b^{1+\lfloor C^*/\epsilon\rfloor}\right)$$
UCS requires memory for the priority queue and explored nodes.
5. Preprocessing Cost
Let:

- V = Number of warehouse locations.
- E = Number of connections.

Building an adjacency list requires:
$$T_p=O(V+E)$$
The overall cost is:

Algorithm	Total Time
BFS	$O(V+E)+O(b^d)$
DFS	$O(V+E)+O(b^m)$
UCS	$O(V+E)+O((V+E)\log V)$ for a graph implementation with a suitable priority queue and decrease-key support

TASK 4 – Analyse Best, Average and Worst-Case Behaviour

Best case, average case and worst case describe how an algorithm performs under different input conditions.

1. Breadth-First Search

- Best Case: The goal is the starting node or the first node explored. Only a small number of nodes are generated.

- Average Case: The goal is located somewhere in the middle of the search tree. BFS explores several nodes before finding it.
- Worst Case: The goal is at the deepest level or is unreachable. BFS explores almost all reachable nodes.

2. Depth-First Search

- Best Case: The goal is present in the first branch explored. The algorithm finds it quickly.
- Average Case: The goal is located in one of the middle branches. DFS explores some branches before reaching it.
- Worst Case: The goal is in the last branch or is unreachable. DFS explores almost the entire search space.

3. Uniform-Cost Search

- Best Case: The goal is the starting node or is reached with a very small path cost.
- Average Case: The goal is found after expanding several locations with lower path costs.
- Worst Case: Many locations have low travel costs, causing the algorithm to explore a large number of nodes before reaching the goal.

Comparison Table

Algorithm	Best Case	Average Case	Worst Case
BFS	Goal at start or first expansion	Goal at intermediate level	Goal unreachable or deepest
DFS	Goal in first branch	Goal in middle branch	Goal in last branch or unreachable
UCS	Very low-cost goal	Several lower-cost nodes explored	Many low-cost alternatives

TASK 5 – Perform Asymptotic Analysis

Asymptotic analysis describes how the time and memory requirements of an algorithm increase when the input size becomes larger.

1. Breadth-First Search

- Best-case time: $\Omega(1)$, when the start is the goal.
- Worst-case time: $O(b^d)$ in the search-tree model.
- Space complexity: $O(b^d)$.
  BFS explores many nodes at each level. Therefore, memory becomes a major limitation when the warehouse grows.

2. Depth-First Search

- Best-case time: $\Omega(1)$.
- Worst-case time: $O(b^m)$.
- Space complexity: $O(bm)$.
  DFS requires less memory than BFS, but it may spend a long time exploring an incorrect branch.

3. Uniform-Cost Search

- Best-case time: $\Omega(1)$.
- Worst-case time: Exponential in the cost bound for a search-tree implementation.

- Graph implementation: $O((V+E)\log V)$ with a suitable priority queue and positive edge costs.
- Space complexity: $O(V+E)$ in a graph implementation storing the frontier and explored information.
  UCS can find the minimum-cost route, but its memory requirement can increase when the number of warehouse locations and connections becomes large.

Comparison for Different Warehouse Sizes

Warehouse States	BFS	DFS	UCS
1,000	Suitable for small searches	Suitable with cycle detection	Suitable
10,000	Memory requirement increases	Uses relatively less memory	Handles weighted routes
100,000	May require high memory	May explore long paths	Requires efficient priority queue
1,000,000+	Memory can become a limitation	Search time can become high	Requires efficient graph storage and search

Conclusion: BFS has high memory requirements, DFS saves memory but may take more time, and UCS is appropriate when travel costs differ. For very large warehouses, efficient data structures and graph preprocessing are important.

TASK 6 – Formulate the Recurrence Relation

For this task, the recursive Depth-First Search algorithm is selected.
A recurrence relation represents the running time of an algorithm in terms of smaller subproblems.
Let:

- m = Remaining search depth.
- b = Branching factor.
- $T(m)$ = Running time of recursive DFS.

At every node, DFS checks the goal and explores its neighbouring nodes.
If the node is not the goal, it can make up to b recursive calls.
The recurrence relation is:

$$T(m)=bT(m-1)+c$$

Where:

- $bT(m-1)$ represents the recursive calls.
- c represents the constant work performed at the current node, such as checking the goal and processing a successor.
- $m-1$ represents the remaining depth after moving to the next node.

Base Case:

$$T(0)=c_0$$

When the depth becomes zero, the algorithm stops expanding further nodes.
Explanation:

The recursive DFS algorithm divides the problem into smaller subproblems by exploring neighbouring locations. Each recursive call explores the next level of the warehouse graph. Therefore, the total running time depends on the branching factor and remaining depth.
This recurrence is a worst-case search-tree model. With a shared visited set in a finite graph, repeated states are avoided and the total graph traversal can instead be bounded by $O(V+E)$.

TASK 7 – Solve the Recurrence Relation

The recurrence relation obtained in Task 6 is:

$$T(m)=bT(m-1)+c$$

We solve this recurrence using the Iteration Method.

Step 1: Expand the recurrence

$$T(m)=bT(m-1)+c$$

Substitute:

$$T(m-1)=bT(m-2)+c$$

Therefore:

$$T(m)=b[bT(m-2)+c]+c$$

$$T(m)=b^2T(m-2)+bc+c$$

Step 2: Expand again

$$T(m)=b^3T(m-3)+b^2c+bc+c$$

Step 3: Continue the process

After m expansions:

$$T(m)=b^mT(0)+c(1+b+b^2+\cdots+b^{m-1})$$

Using the geometric series:

$$T(m)=b^mT(0)+c\left(\frac{b^m-1}{b-1}\right)$$

For $b>1$:

$$\boxed{T(m)=O(b^m)}$$

Step 4: Interpretation

1. The running time increases exponentially as the branching factor increases.
2. When the warehouse becomes deeper, DFS may need to explore many additional locations.
3. Recursive DFS can use less memory than BFS because it mainly stores the current search path and pending alternatives.
4. Therefore, DFS is memory-efficient in many search-tree situations, but its execution time may become high.

Extension to Iterative Deepening Search

Iterative Deepening Search repeatedly performs depth-limited DFS with increasing depth limits.

The search limits are:

$$0,1,2,3,\ldots,d$$

Its total work is:

$$1+(d+1)b+(d)b^2+\cdots+b^d$$

Its time complexity is:

$$ O(b^d) $$

Its space complexity is:

$$ O(bd) $$

Conclusion: Iterative Deepening Search combines the low memory requirement of DFS with the ability to find a shallowest solution like BFS when all step costs are equal.

TASK 8 – Implement and Test the Algorithms

This task mainly requires practical implementation, but the following theory explains the testing procedure.

The three selected algorithms are Breadth-First Search, Depth-First Search and Uniform-Cost Search.

Testing Procedure

1. Generate warehouse graphs containing approximately 1,000, 10,000, 100,000 and 1,000,000 locations.
2. Assign connections between warehouse locations.
3. Select a starting location and a destination.
4. Run all three algorithms using the same graph and starting and goal locations.
5. Record the generated nodes, expanded nodes, path cost and execution time.
6. Repeat the experiments using the same random seeds to ensure fair comparison.

Types of Test Cases

Test Case	Input Condition	Expected Behaviour
Test 1	Goal is reachable	All algorithms should find a route.
Test 2	Goal is unreachable	All algorithms should return failure.
Test 3	Start = Goal	All algorithms should return the starting location immediately.

Expected Behaviour

- BFS should find the shallowest route when all movements have equal costs.
- DFS should find a route if one exists and the search is complete with appropriate cycle handling.
- UCS should find the minimum-cost route when all step costs are positive.

Conclusion: Testing verifies whether the algorithms work correctly and whether their actual performance agrees with the theoretical analysis.

Note: This is the theory and expected behaviour. Actual Python source code and execution outputs must be submitted separately as experimental evidence.

TASK 9 – Record Experimental Performance

Performance analysis measures how efficiently the algorithms operate for different warehouse sizes.

The following parameters should be recorded.

1. Nodes Generated: Total number of nodes created during the search.
2. Nodes Expanded: Number of nodes removed from the frontier and explored.

3. Maximum Frontier Size: Maximum number of nodes stored in the queue, stack or priority queue.
4. Solution Depth: Number of movements required to reach the destination.
5. Path Cost: Total travel time required to reach the goal.
6. Execution Time: Total time taken by the algorithm to complete the search.

Performance Recording Table

Use the following table to record the actual experimental readings.

States	Algorithm	Nodes Generated	Nodes Expanded	Max Frontier
1,000	BFS	To measure	To measure	To measure
1,000	DFS	To measure	To measure	To measure
1,000	UCS	To measure	To measure	To measure
10,000	BFS	To measure	To measure	To measure
10,000	DFS	To measure	To measure	To measure
10,000	UCS	To measure	To measure	To measure
100,000	All three	To measure	To measure	To measure
1,000,000	All three	To measure	To measure	To measure

The execution time should be measured over at least five runs and the average should be calculated.

$$ Average\ Time=\frac{\text{Sum of 5 execution times}}{5} $$

Graphical Representation

The following graphs should be prepared:

- Graph 1: Warehouse size vs nodes generated.
- Graph 2: Warehouse size vs nodes expanded.
- Graph 3: Warehouse size vs maximum frontier size.
- Graph 4: Warehouse size vs average execution time.

Conclusion: Experimental performance analysis helps identify the most efficient algorithm for different warehouse sizes. It also helps understand the relationship between theoretical complexity and actual execution behaviour.

Actual performance values and graphs cannot be truthfully supplied without running the algorithms on the specified graphs.

TASK 10 – Compare Theory with Experimental Behaviour

Theoretical analysis predicts the performance of an algorithm, whereas experimental analysis measures its actual performance.

1. Breadth-First Search

Theoretical analysis shows that BFS requires considerable memory because it stores all generated nodes at the current frontier.

In experiments, the number of generated and expanded nodes is expected to increase as the warehouse size increases. Therefore, its memory requirement may become a limitation in large warehouses.

2. Depth-First Search

Theoretical analysis shows that DFS requires less memory than BFS because it explores one branch at a time.

Experimental results are expected to show a smaller frontier in many cases. However, DFS may take more time if the goal is located in a different or deeper branch.

3. Uniform-Cost Search

Theoretical analysis shows that UCS expands nodes based on their path costs.

Experimental results are expected to show that UCS may explore additional nodes when many low-cost alternatives are available. However, it can provide the minimum-cost route.

Comparison Table

Parameter	BFS	DFS	UCS
Search Method	Level-wise	Depth-wise	Lowest-cost first
Memory Usage	High	Low in tree search	High
Shortest Path	Equal-cost edges	Not guaranteed	Minimum-cost path
Weighted Graph	Not suitable for cost optimality	Not suitable for cost optimality	Suitable
Large Graph	Memory limitation	May take more time	Depends on graph costs

Reasons for Differences Between Theory and Experiments

1. Different warehouse graph structures may affect the number of nodes explored.
2. Duplicate locations may increase operations if cycle detection is not used.
3. Programming language overhead may affect execution time.
4. The order in which neighbouring nodes are explored may change the search results.
5. Computer memory and processor speed may affect actual performance.

Conclusion: Theoretical and experimental results should be compared to understand the actual efficiency of each algorithm. The final decision should consider execution time, memory requirements and path quality.

TASK 11 – Recommend and Defend the Final Design

Recommended Algorithm: Uniform-Cost Search

Uniform-Cost Search is recommended for the warehouse picking robot because different warehouse paths may have different travel-time costs.

Reasons for Selecting UCS

1. UCS selects the next location based on the lowest accumulated travel cost.
2. It finds the minimum-cost route when all movement costs are positive.
3. It considers different travel times between warehouse locations.
4. It avoids selecting a longer or more expensive route when a cheaper route is available.
5. It is suitable for warehouse environments where travel efficiency is important.

Comparison with Other Algorithms

BFS: BFS finds the shortest route in terms of the number of movements when all edges have equal costs. However, it may not find the minimum travel-time route when movement costs differ.

DFS: DFS requires less memory in many tree-search situations, but it may explore unnecessary paths and does not guarantee an optimal route.

UCS: UCS considers the total travel cost and selects the lowest-cost path. Therefore, it is more appropriate for the given weighted warehouse environment.

Limitations

- UCS requires more memory for storing frontier and explored nodes.
- Priority queue operations increase processing requirements.
- The warehouse graph must be updated when new paths or obstacles appear.
- Preprocessing is required to build the graph and adjacency lists.

Final Recommendation:

Uniform-Cost Search is the most appropriate choice for the warehouse picking robot because it focuses on minimizing total travel time rather than simply minimizing the number of movements. It provides a reliable minimum-cost route and is suitable for weighted warehouse environments.

TASK 12 – Re-evaluate and Adapt the Design

A. Impact Analysis

In the original design, the robot knows its exact starting location. After the QR-code reader fails, the robot does not know its exact location and only knows that it may be at one of \(k\) possible locations.

The following changes occur:

1. The environment becomes partially observable.
2. The robot cannot identify its exact starting position.
3. Its knowledge about possible locations becomes part of the search state.
4. The robot must find a route to the charging dock regardless of its actual starting location.

Therefore, the original assumption of a fully known starting state is no longer valid.

B. Evaluation of the Original Design (2 Marks)

The original search algorithms can still be used, but the state representation must be modified.

Earlier, a state represented one warehouse location. Now, a state represents a set of possible warehouse locations. This is called a Belief State.

For example:

- If \(k=2\), the robot may be at two possible locations.
- If \(k=4\), the robot may be at four possible locations.
- If \(k=8\), the robot may be at eight possible locations.

The number of possible non-empty belief states is:

\[2^k-1 \]

Possible Locations (k)	Possible Belief States
2	3
4	15

8	255

As the number of possible locations increases, the search space also increases.
Therefore, directly applying the original search algorithm without modification may become inefficient.

C. Adaptation or Redesign

Recommended Approach: Modify the Original Design Using Belief-State Search and Local Sensing.
The robot should maintain a set of possible locations instead of assuming one exact location.
The revised working process is:

1. The robot identifies all possible locations where it may be present.
2. It stores these locations as a belief state.
3. It uses available sensors to collect information about its surroundings.
4. The sensor reduces the number of possible locations.
5. The robot searches for a route to the charging dock.
6. It follows a conditional plan depending on the sensor information.



Effect on Complexity:
The original search space contains \(V\) individual locations.
The modified belief-state search can contain up to:
\[2^V-1 \]
non-empty belief states in the general case.

This can significantly increase the search space. However, local sensing helps reduce uncertainty and makes the search more efficient.

D. Evidence-Based Defence

To evaluate the modified design, consider three experiments with different numbers of possible starting locations.

Experiment 1: Without Local Sensor

The robot maintains all possible locations in its belief state.

Experiment 2: With Local Sensor

The robot uses a local sensor to identify its actual location and reduces uncertainty.

Theoretical Comparison:

Possible Locations	Belief States Without Sensor	Remaining Candidates With Perfect Sensor
2	3	1
4	15	1
8	255	1

The table shows the theoretical number of possible non-empty belief states and the number of candidates after perfect localization.

Expected Results

1. Without a sensor, the robot must consider multiple possible starting locations.
2. With a local sensor, the robot can identify its exact location.
3. The number of possible starting locations reduces to one after perfect sensing.
4. Therefore, sensing reduces uncertainty and can reduce the search effort.

The exact plan length depends on the warehouse graph, sensor accuracy and available movements. It must be measured experimentally rather than assumed.